AI Checkout Copilot

Product Strategy, Scope, and Decision Document

---

1. The Problem & Its Significance

The final step of the e-commerce funnel—the shipping and payment screen—is historically the highest point of friction. Cart abandonment averages 70%, with "unexpected shipping costs" consistently cited as the primary driver.

Currently, the checkout experience is entirely static. A customer purchasing a 950 dollar premium snowboard sees the exact same uncontextualized "15 dollars Express Shipping" text as someone buying a 10 dollars pair of socks. This lack of context forces the user to calculate the value tradeoff themselves, leading to decision fatigue and abandonment. Furthermore, merchants lose critical last-minute Average Order Value (AOV) expansion opportunities because traditional upselling widgets are blind to the immediate context of the cart's value and intent.

2. Target Users & The Status Quo

This product serves a dual-sided user base:

- **The Merchant (The Buyer of the App):** Premium, high-ticket Shopify Plus brands. Their current experience involves relying on static shipping tables and aggressive, pop-up-based upsell apps that interrupt the user journey and cheapen the brand feel.
- **The Shopper (The End User):** High-intent buyers. Their current experience at checkout is isolated and transactional. They are presented with raw costs rather than value propositions, making shipping fees feel like a penalty rather than a service.

3. The Solution & Core User Journey

AI Checkout Copilot is a context-aware advisory widget embedded natively into the Shopify checkout flow.

The Core Journey:

1. The user enters their shipping address, triggering the generation of delivery options.
2. The Copilot silently captures the live cart data (item names, subtotal, tax, and available shipping tiers).
3. Without requiring a click or prompt from the user, the Copilot generates a concise, 3-sentence recommendation. It explicitly links the value of the items (e.g., the 950 dollar snowboard) to the logic of upgrading shipping (e.g., protecting the investment).

4. The Copilot organically suggests a highly relevant cross-sell (e.g., bindings or wax) based purely on the detected context.

4. Key Product Decisions

1. **Native UI Extension over App Blocks:** We explicitly chose to build within Shopify's restrictive Checkout UI Web Worker sandbox rather than a traditional App Block. **Reasoning:** Checkout is a high-trust environment. Redirecting users or injecting iframe-based popups degrades trust and conversion. Natively rendering Preact components ensures the UI feels like a first-party Shopify feature.
2. **Contextual Push over Conversational Pull:** We engineered the system prompt to explicitly forbid the AI from asking follow-up questions. **Reasoning:** Checkout is not the place for a chatbot. The goal is conversion, not conversation. The AI must act as an assertive advisor, delivering an immediate conclusion without requiring the user to type.

5. Scope Constraints: What We Chose NOT to Build

- **A Chat Interface:** We actively scoped out a text input box. While a chatbot sounds impressive for an AI hackathon, requiring a user to type questions at the literal point of purchase introduces fatal friction. We opted for a zero-click, auto-generating insight model.
- **Backend Inventory Syncing:** We chose not to build a database to map product SKUs to potential upsells. **Reasoning:** It introduces massive latency. Instead, we relied entirely on the LLM's zero-shot reasoning capabilities to "hallucinate" logical accessory pairings (e.g., knowing a snowboard needs bindings) purely based on the live string names of the cart items.

6. Tradeoffs & Resolutions

Tradeoff 1: API Latency vs. Reactive State Updates Shopify's checkout state is highly reactive; clicking a shipping radio button fires multiple rapid state updates. Passing every update to the Gemini API would result in 429 Rate Limit errors, high costs, and a jittery UI. **Resolution:** We built a localized `useRef` memory cache. The frontend stringifies the core cart variables into a snapshot. The API is only called if the new snapshot explicitly differs from the cached snapshot, permanently bypassing rate limits and ensuring a smooth UI.

Tradeoff 2: Data Resiliency vs. Code Simplicity Shopify's checkout API schemas shift frequently, with item data sometimes nested under `.current` signals and sometimes under `.value` objects. Writing a simple, single-path data extractor caused fatal sandbox crashes during testing. **Resolution:** We sacrificed code brevity to build a "Wide Net" data parser. The frontend uses aggressive logical fallbacks (`shopify.lines?.current || shopify.lines?.value`) to dynamically traverse the schema, ensuring the AI never receives a blank payload, thus preventing LLM hallucinations.